

(continued)

subtype of `Y`, then `List[X]` is a subtype of `List[Y]`). We could leave out the `+` in front of the `A`, which would make `List` *invariant* in that type parameter.

But notice now that `Nil` extends `List[Nothing]`. `Nothing` is a subtype of all types, which means that in conjunction with the *variance annotation*, `Nil` can be considered a `List[Int]`, a `List[Double]`, and so on, exactly as we want.

These concerns about variance aren't very important for the present discussion and are more of an artifact of how Scala encodes data constructors via subtyping, so don't worry if this is not completely clear right now. It's certainly possible to write code without using variance annotations at all, and function signatures are sometimes simpler (whereas type inference often gets worse). We'll use variance annotations throughout this book where it's convenient to do so, but you should feel free to experiment with both approaches.

3.2 *Pattern matching*

Let's look in detail at the functions `sum` and `product`, which we place in the object `List`, sometimes called the *companion object* to `List` (see sidebar). Both these definitions make use of pattern matching:

```
def sum(ints: List[Int]): Int = ints match {
  case Nil => 0
  case Cons(x,xs) => x + sum(xs)
}

def product(ds: List[Double]): Double = ds match {
  case Nil => 1.0
  case Cons(0.0, _) => 0.0
  case Cons(x,xs) => x * product(xs)
}
```

As you might expect, the `sum` function states that the sum of an empty list is 0, and the sum of a nonempty list is the first element, `x`, plus the sum of the remaining elements, `xs`.³ Likewise the `product` definition states that the product of an empty list is 1.0, the product of any list starting with 0.0 is 0.0, and the product of any other nonempty list is the first element multiplied by the product of the remaining elements. Note that these are recursive definitions, which are common when writing functions that operate over recursive data types like `List` (which refers to itself recursively in its `Cons` data constructor).

Pattern matching works a bit like a fancy `switch` statement that may descend into the structure of the expression it examines and extract subexpressions of that

³ We could call `x` and `xs` anything there, but it's a common convention to use `xs`, `ys`, `as`, or `bs` as variable names for a sequence of some sort, and `x`, `y`, `z`, `a`, or `b` as the name for a single element of a sequence. Another common naming convention is `h` for the first element of a list (the *head* of the list), `t` for the remaining elements (the *tail*), and `l` for an entire list.

structure. It's introduced with an expression (the *target* or *scrutinee*) like `ds`, followed by the keyword `match`, and a `{}`-wrapped sequence of *cases*. Each case in the match consists of a *pattern* (like `Cons(x, xs)`) to the left of the `=>` and a *result* (like `x * product(xs)`) to the right of the `=>`. If the target *matches* the pattern in a case (discussed next), the result of that case becomes the result of the entire match expression. If multiple patterns match the target, Scala chooses the first matching case.

Companion objects in Scala

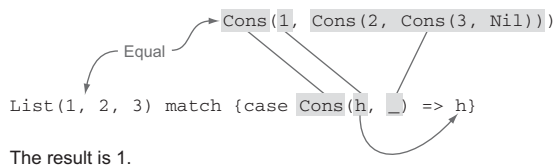
We'll often declare a *companion object* in addition to our data type and its data constructors. This is just an object with the same name as the data type (in this case `List`) where we put various convenience functions for creating or working with values of the data type.

If, for instance, we wanted a function `def fill[A](n: Int, a: A): List[A]` that created a `List` with `n` copies of the element `a`, the `List` companion object would be a good place for it. Companion objects are more of a convention in Scala.⁴ We could have called this module `Foo` if we wanted, but calling it `List` makes it clear that the module contains functions relevant to working with lists.

Let's look at a few more examples of pattern matching:

- `List(1,2,3) match { case _ => 42 }` results in 42. Here we're using a variable pattern, `_`, which matches any expression. We could say `x` or `foo` instead of `_`, but we usually use `_` to indicate a variable whose value we ignore in the result of the case.⁵
- `List(1,2,3) match { case Cons(h, _) => h }` results in 1. Here we're using a data constructor pattern in conjunction with variables to *capture* or *bind* a sub-expression of the target.
- `List(1,2,3) match { case Cons(_, t) => t }` results in `List(2,3)`.
- `List(1,2,3) match { case Nil => 42 }` results in a `MatchError` at runtime. A `MatchError` indicates that none of the cases in a match expression matched the target.

Matching a list



⁴ There is some special support for them in the language that isn't really relevant for our purposes.

⁵ The `_` variable pattern is treated somewhat specially in that it may be mentioned multiple times in the pattern to ignore multiple parts of the target.

What determines if a pattern matches an expression? A pattern may contain *literals* like 3 or "hi"; *variables* like x and xs, which match anything, indicated by an identifier starting with a lowercase letter or underscore; and data constructors like Cons(x, xs) and Nil, which match only values of the corresponding form. (Nil as a pattern matches only the value Nil, and Cons(h, t) or Cons(x, xs) as a pattern only match Cons values.) These components of a pattern may be nested arbitrarily—Cons(x1, Cons(x2, Nil)) and Cons(y1, Cons(y2, Cons(y3, _))) are valid patterns. A pattern *matches* the target if there exists an assignment of variables in the pattern to subexpressions of the target that make it *structurally equivalent* to the target. The resulting expression for a matching case will then have access to these variable assignments in its local scope.



EXERCISE 3.1

What will be the result of the following match expression?

```
val x = List(1,2,3,4,5) match {
  case Cons(x, Cons(2, Cons(4, _))) => x
  case Nil => 42
  case Cons(x, Cons(y, Cons(3, Cons(4, _)))) => x + y
  case Cons(h, t) => h + sum(t)
  case _ => 101
}
```

You're strongly encouraged to try experimenting with pattern matching in the REPL to get a sense for how it behaves.

Variadic functions in Scala

The function apply in the object List is a *variadic function*, meaning it accepts zero or more arguments of type A:

```
def apply[A](as: A*): List[A] =
  if (as.isEmpty) Nil
  else Cons(as.head, apply(as.tail: _*))
```

For data types, it's a common idiom to have a variadic apply method in the companion object to conveniently construct instances of the data type. By calling this function apply and placing it in the companion object, we can invoke it with syntax like List(1,2,3,4) or List("hi", "bye"), with as many values as we want separated by commas (we sometimes call this the *list literal* or just *literal* syntax).

Variadic functions are just providing a little syntactic sugar for creating and passing a Seq of elements explicitly. Seq is the interface in Scala's collections library implemented by sequence-like data structures such as lists, queues, and vectors. Inside apply, the argument as will be bound to a Seq[A] (documentation at <http://mng.bz/f4k9>), which has the functions head (returns the first element) and tail (returns all elements but the first). The special _* type annotation allows us to pass a Seq to a variadic method.